Day 128: Anomaly Detection With Autoencoders: Real Reconstruction-Error Thresholds

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 19, Day 3 | Complete study guide

Watch: 88 - Applications of Autoencoders - Anomaly Detection (DigitalSreeni) | Anomaly Detection Using Autoencoders | Tensorflow | Keras (MLWorks)

Script: `day128_anomaly_detection.py` · every number in this guide is copied by program from `day128_run_output.txt`, the real output of that script.

Train an autoencoder on normal data only, and score everything by how badly it is rebuilt. Tested honestly, the idea works only when anomalies are harder to rebuild than normal data. Holding out MNIST 9s, the autoencoder barely beats chance (0.5490 ROC AUC), and the plain nearest-neighbour distance does better (0.6754); held-out 1s are rebuilt *better* than normal digits (0.2375). On real tumours it works well (0.9288) but no better than PCA (0.9366). Thresholds set on held-out normal data keep their promised false-alarm rate; a few anomalies leaking into the training data ruin the detector.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

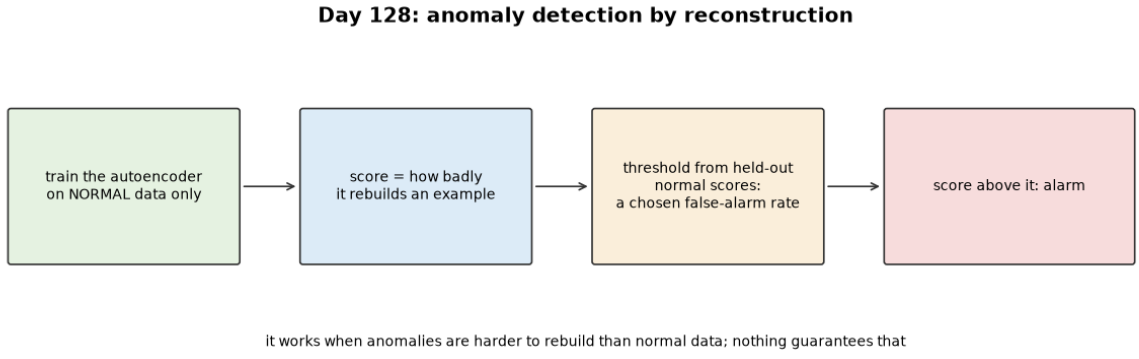
Anomaly detection flags examples unlike the normal ones. Here an autoencoder (Day 126) is trained only on normal data; its **reconstruction error** on a new example is the **anomaly score**. A **threshold** turns scores into alarms.

Only normal data is needed to train and to set the threshold; labelled anomalies are used only to judge. Day 117's **ROC AUC** (the chance an anomaly outscores a normal example) and **average precision** measure the scores.

REAL-WORLD SCENARIO

A factory camera trained on thousands of good parts, with no photos of the rare faults: a part it cannot 'redraw' from memory is sent for inspection. Whether that works depends on whether faults really look unusual to the camera.

SYSTEM DIAGRAM



Train on normal data, score by reconstruction error, set the threshold on held-out normal data.

Today's part for the week's build

`day128_exercise.py` is the three scores, the threshold and its rates, checked by hand. Its real output:

ACTUAL OUTPUT

```
threshold for 20% false alarms on [1, 2, 2.5, 3, 4, 3.5]: 3.50 (by hand: the 80th percentile, 3.5)
recall 1.000 (2 of 2), false alarms 0.167 (1 of 6), precision 0.667 (2 of 3)
distance from (3, 0) to the nearest of (0, 0) and (3, 4): 3.0000 (by hand 3)
PCA score, 1 component, of a point on the line: 0.00e+00; of (1, 0): 0.8000 (by hand 4/5 = 0.8)
```

Data, all real: MNIST (cached by Day 126) with one digit held out as the anomaly; and the Wisconsin breast-cancer measurements (bundled with scikit-learn), with the model trained on benign tumours only and malignant ones as the anomalies. Day 126's network, Day 123's PCA and Day 117's metrics are imported.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. By hand: a score, a threshold, and what it costs

DEFINITION — start here, no prior knowledge assumed

Eight scores, two of them anomalies. The threshold is set from the six normal scores alone, for a 20% false-alarm rate.

REAL-WORLD SCENARIO

A smoke alarm's sensitivity set by how often it may ring when someone is only making toast.

MATHEMATICS

threshold t = the $(1 - \text{false-alarm rate})$ quantile of normal scores; alarm if score $> t$

recall = flagged anomalies / anomalies; false alarms = flagged normal / normal; precision = flagged anomalies / flagged

IMPLEMENTATION

```
def ae_score(net, X, acts, kind="bce"):
    return ((d126.reconstruct(net, X, acts, kind) - X) ** 2).sum(1)
def pca_score(X, pca, k):
    mu, V = pca
    return ((d123.reconstruct(d123.transform(X, mu, V, k), mu, V, k) - X) ** 2).sum(1)
def nn_score(X, ref, chunk=2000):
    return np.concatenate([np.sqrt(np.maximum(d119.sqdist(X[s:s + chunk], ref), 0).min(1)) for s
        in range(0, len(X), chunk)])
def threshold(normal_scores, false_alarm):
    return float(np.quantile(normal_scores, 1 - false_alarm))
def alarm_stats(scores, y, t):
    flag = scores > t; tp = int((flag & (y == 1)).sum()); fp = int((flag & (y == 0)).sum())
    return {"recall": tp / max((y == 1).sum(), 1), "false alarms": fp / max((y == 0).sum(), 1),
        "precision": tp / max(flag.sum(), 1)}
```

ACTUAL OUTPUT

```
scores [1.0, 2.0, 2.5, 3.0, 4.0, 9.0, 3.5, 8.0], anomalies at 9.0 and 8.0; threshold for a 20%
    false-alarm rate on the six normal scores: 3.50
flagged: [4.0, 9.0, 8.0]; recall 1.000, false alarms 0.167, precision 0.667
ROC AUC 1.0000: by hand, both anomalies score above all six normal examples, so 12 of 12 pairs are
    ordered right
Actual output from this lesson's run.
```

What it means

The 80th percentile of the six normal scores is 3.50. Above it: 4.0 (a false alarm) and both anomalies, so recall 1.000, false alarms 0.167 and precision 0.667. The ROC AUC is 1.0000: every anomaly outscores every normal example, even though one normal example is flagged. The AUC judges the ordering; the threshold decides what that ordering costs.

WHY CHOOSE THIS?

The threshold needs normal data only.

WHY NOT THIS? (TRADEOFFS)

Its false-alarm rate is a promise about normal data, not about how many anomalies it catches.

2. One digit held out

DEFINITION — start here, no prior knowledge assumed

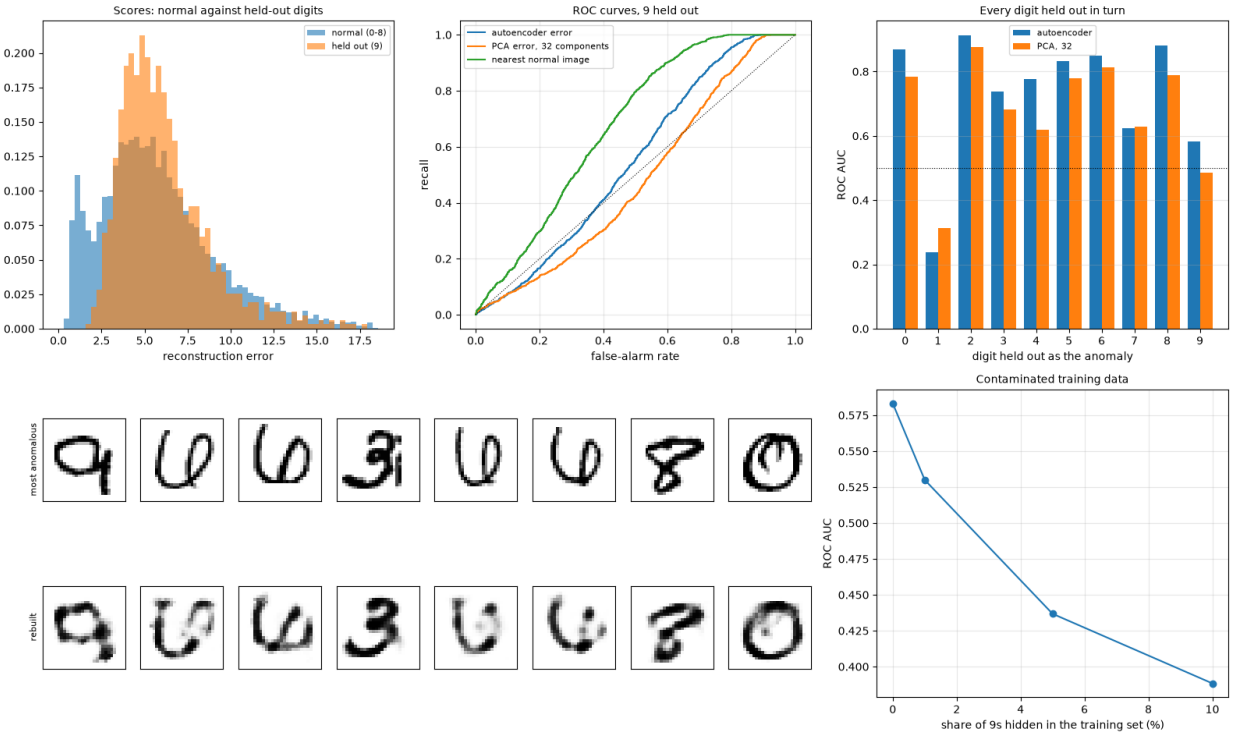
Trained on 20,000 images of the digits 0 to 8; the test set's 9s are the anomalies. Three scores: the autoencoder's error, PCA's error with 32 components, and the distance to the nearest of 10,000 normal training images.

REAL-WORLD SCENARIO

Training a guard on everyone in the building except one department, then seeing whether that department looks like strangers.

ACTUAL OUTPUT

```
anomalies: 1009 of 10000 test images (0.101); the autoencoder trained in 84 s
  score                         ROC AUC  average precision
  autoencoder error              0.5490             0.1053
  PCA error, 32 components       0.4825             0.0958
  nearest normal image           0.6754             0.1519
Actual output from this lesson's run.
```



Top: scores of normal and held-out digits; ROC curves with 9 held out; AUC for every held-out digit. Bottom: the eight most anomalous test images and their reconstructions; ROC AUC as 9s leak into the training data.

What it means

The autoencoder barely beats chance (0.5490), PCA does not (0.4825), and the plain nearest-neighbour distance is best (0.6754). A 9 differs from a 4 or a 7 by a small change, which is probably why an autoencoder trained on those rebuilds 9s almost as well as its own digits: the two score histograms overlap almost completely. The most anomalous test images in the figure are mostly oddly written 6s, not 9s.

WHY CHOOSE THIS?

No anomaly examples needed to build the detector.

WHY NOT THIS? (TRADEOFFS)

It finds 'hard to rebuild', which is not the same as 'the kind of anomaly you care about'.

3. Every digit held out in turn

DEFINITION — start here, no prior knowledge assumed

Ten detectors, each trained on 10,000 images of nine digits for 10 epochs, against PCA.

REAL-WORLD SCENARIO

Testing the guard once for each department in turn.

ACTUAL OUTPUT

```
   held out   AE AUC  PCA AUC  mean error, normal  mean error, held out
          0   0.8701   0.7846                8.37                 16.85
          1   0.2375   0.3126                9.37                  5.93
          2   0.9123   0.8755                7.87                 18.06
          3   0.7368   0.6816                8.34                 12.48
          4   0.7773   0.6189                8.18                 12.69
          5   0.8320   0.7803                8.17                 14.39
          6   0.8486   0.8143                8.18                 15.88
          7   0.6239   0.6286                8.55                 10.81
          8   0.8820   0.7883                8.00                 15.90
          9   0.5832   0.4860                8.61                  9.67
an AUC below 0.5 means the held-out digit is rebuilt BETTER than the normal ones on average
Actual output from this lesson's run.
```

What it means

The answer depends entirely on which digit is missing: from 0.9123 (held-out 2s) to 0.2375 (held-out 1s). A 1 is a single stroke, easier to rebuild than the digits the network was trained on: its mean error (5.93) is below the normal digits' (9.37), so the detector ranks 1s as the *most* normal images. The autoencoder beats PCA on eight of the ten digits.

WHY CHOOSE THIS?

A fair test across many kinds of anomaly.

WHY NOT THIS? (TRADEOFFS)

Shorter training (10,000 images, 10 epochs) than section 2, to keep ten runs affordable.

4. Thresholds from normal data

DEFINITION — start here, no prior knowledge assumed

Thresholds set on 5,000 normal images the network never trained on, for false-alarm rates from 1% to 20%, judged on the test set.

REAL-WORLD SCENARIO

Calibrating a scale with known weights it has never been used on before.

ACTUAL OUTPUT

```
   false alarms aimed at  threshold  false alarms got   recall  precision
                    0.01      17.84            0.0065   0.0079     0.1212
                    0.05      12.93            0.0409   0.0327     0.0823
                    0.10      10.84            0.0863   0.0614     0.0740
                    0.20       8.63            0.1786   0.1427     0.0823
  the same 5% target set on the network's own training images instead: threshold 10.86, false
      alarms got 0.0858
Actual output from this lesson's run.
```

What it means

The false-alarm rates delivered are close to those aimed at (0.0409 for 5%, 0.0863 for 10%): the threshold keeps its promise about normal data. Recall is poor because 9s are hard to detect at all (0.0327 at 5%). Setting the same 5% threshold on the network's own training images gives 10.86 instead of 12.93 and almost twice the false alarms (0.0858): training images are rebuilt unusually well, so they make every new image look suspicious.

WHY CHOOSE THIS?

A false-alarm rate you can promise in advance.

WHY NOT THIS? (TRADEOFFS)

It needs normal data the model has not trained on.

5. Contaminated training data

DEFINITION — start here, no prior knowledge assumed

The 'normal' training set secretly contains some 9s: 0%, 1%, 5% and 10% of 10,000 images.

REAL-WORLD SCENARIO

A guard who has been shown a few intruders and told they belong.

ACTUAL OUTPUT

```
     0% of the 10,000 training images are 9s (   0): ROC AUC 0.5832
     1% of the 10,000 training images are 9s ( 100): ROC AUC 0.5300
     5% of the 10,000 training images are 9s ( 500): ROC AUC 0.4368
    10% of the 10,000 training images are 9s (1000): ROC AUC 0.3884
Actual output from this lesson's run.
```

What it means

Even 1% contamination lowers the AUC (0.5832 to 0.5300); at 5% it falls below chance (0.4368). Once the network has seen 9s as normal, it learns to rebuild them, and they stop standing out. Unsupervised anomaly detection depends on a clean idea of 'normal'.

WHY CHOOSE THIS?

Shows the method's main risk, measured.

WHY NOT THIS? (TRADEOFFS)

One held-out digit, which was already hard; easier anomalies may survive more contamination.

6. Tumours

DEFINITION — start here, no prior knowledge assumed

A small autoencoder (30-16-4-16-30) trained on 200 benign tumours, five seeds; scored on 157 benign and 212 malignant. PCA with 4 components and the nearest benign training tumour for comparison.

REAL-WORLD SCENARIO

A pathologist who has only ever seen healthy tissue, asked to flag anything that looks different.

ACTUAL OUTPUT

```
training: 200 benign; test: 157 benign and 212 malignant
  autoencoder 30-16-4-16-30, 5 seeds: ROC AUC 0.9288 (from 0.9137 to 0.9436)
  PCA error, 4 components:          ROC AUC 0.9366
  nearest benign training tumour:  ROC AUC 0.9450
no malignant tumour was used to train any of the three; labels are used only to judge
Actual output from this lesson's run.
```

What it means

Here the idea works: malignant tumours really are unlike benign ones in these measurements, and the autoencoder reaches 0.9288 (from 0.9137 to 0.9436 across seeds) without seeing a single malignant example. But the simpler scores do as well or slightly better: PCA 0.9366, nearest neighbour 0.9450. With 30 measurements and 200 examples there is probably little curved structure for the network to exploit.

WHY CHOOSE THIS?

Real anomalies found with no anomaly labels.

WHY NOT THIS? (TRADEOFFS)

Try PCA and nearest neighbours first: they matched it here.

Interview preparation — technical questions

Q: How does autoencoder anomaly detection work?

A: Train on normal data; score by reconstruction error; alarm above a threshold.

Q: How do you set the threshold?

A: From held-out normal data at a chosen false-alarm rate: 5% aimed, 0.0409 delivered.

Q: Why not set it on the training data?

A: Training examples are rebuilt too well: the same target gave 0.0858 false alarms.

Q: What baseline would you compare with?

A: PCA error and nearest-neighbour distance: the latter won on 9s (0.6754) and tumours (0.9450).

Q: What breaks it?

A: Anomalies in the training data (5% gave AUC 0.4368) and anomalies that are easy to rebuild (1s: 0.2375).

Interview preparation — theoretical questions

Q: Why can an anomaly score below normal data?

A: Simple shapes (a 1) are easier to rebuild than the training digits themselves.

Q: What does ROC AUC measure here?

A: The chance an anomaly outscores a normal example, at every threshold at once.

Q: Why is average precision low for 9s?

A: Anomalies are 10% of the test set and barely separated: 0.1053 against 0.101 by chance.

Q: When does a network beat PCA?

A: When normal data has curved structure: on digits (8 of 10), not on 30 tumour measurements.

Key takeaway and what's next

KEY TAKEAWAY

Reconstruction error is a detector of 'hard to rebuild', which is an anomaly only when anomalies happen to be hard to rebuild. On tumours they were, and the method worked; on digits it ranged from good to worse than chance, depending on which digit was missing. Set thresholds on held-out normal data, keep anomalies out of training, and always compare with PCA and nearest neighbours.

- 9s held out: autoencoder 0.5490, PCA 0.4825, nearest neighbour 0.6754.
- Every digit: from 0.9123 (2s) to 0.2375 (1s).

- 5% threshold: 0.0409 false alarms from held-out data, 0.0858 from training data.
- Tumours: autoencoder 0.9288, PCA 0.9366, nearest neighbour 0.9450.

Suggested watch order, if starting from zero

- 1. 88 - Applications of Autoencoders - Anomaly Detection (DigitalSreeni) — anomaly detection with an autoencoder, in code.
- 2. Anomaly Detection Using Autoencoders | Tensorflow | Keras (MLWorks) — reconstruction-error thresholds, step by step.

Day 129 preview

- **Variational autoencoders**: codes as random draws, and the reparameterization trick that lets gradients through them.